

# Opus Devia

## Tiers, XP Economy & Billing Framework

Complete Architecture Deep-Dive

---

Generated: July 10, 2026

Source: kin workspace -- supabase/functions + migrations + opus-devia/src

### Table of Contents

1. Executive Summary
2. The Four Tiers -- Full Comparison
3. Tier-Dependent Infrastructure (Storage, Rollover)
4. XP Economy -- Data Model
5. XP Economy -- RPC Functions (Reserve / Deduct / Credit / Rollover)
6. billing-manager Edge Function -- XP Gateway
7. XP Reservations & Dead Letter Queue
8. Billing Tier Governance (Feature Gating & Token Budgets)
9. Model Routing & Provider Architecture
10. Frontend XP Integration (Profile, Home, Settings)
11. Performance Metrics & Community XP Sync
12. End-to-End XP Lifecycle (Walkthrough)
13. Key Architectural Decisions & Trade-offs

### 1. Executive Summary

Opus Devia implements a four-tier freemium model (Free, Builder, Operator, Founder) with XP (Experience Points) as the universal currency that gates every AI feature. XP is not cosmetic -- it is the literal fuel that powers AI interactions. Each message to the Mentor, Assistant, or background analysis engine costs XP from one of three pools (earned, purchased, rollover). The system uses a dual-write reservation pattern (preflight -> finalize/cancel) to prevent race conditions and overspend, backed by atomic PostgreSQL RPCs running as SECURITY DEFINER. A monthly cron-triggered rollover process transfers a tier-dependent percentage of unused earned XP into the rollover pool, with Free tier users forfeiting 100%.

Key numbers: 17 features are gated by tier, 6 AI model providers are routed dynamically, 4 XP pools are tracked per user (earned, purchased, rollover, reserved), and the entire system outputs immutable transaction logs with idempotency keys for auditability.

## 2. The Four Tiers -- Full Comparison

Tiers are stored as a PostgreSQL enum (`user\_tier`) on the `users` table. Each tier unlocks progressively more features, higher token budgets, larger storage limits, larger rollover percentages, and higher XP earning multipliers.

| Attribute             | Free   | Builder    | Operator | Founder   |
|-----------------------|--------|------------|----------|-----------|
| Monthly Token Budget  | 50,000 | 150,000    | 400,000  | 1,000,000 |
| Storage Limit         | 50 MB  | 500 MB     | 2 GB     | 10 GB     |
| Rollover %            | 0%     | 25%        | 50%      | 100%      |
| XP Multiplier         | 1x     | 1.5x       | 2x       | 3x        |
| Mentor Support        | No     | Yes        | Yes      | Yes       |
| Voice I/O             | No     | Yes        | Yes      | Yes       |
| Image Upload          | No     | No         | Yes      | Yes       |
| Roadmap Generation    | No     | No         | Yes      | Yes       |
| Roadmap Recalibration | No     | No         | Yes      | Yes       |
| Weekly/Monthly Review | No     | Daily only | All      | All       |
| Chat Pattern Analysis | No     | No         | Yes      | Yes       |
| Memory Tagging (auto) | No     | Yes        | Yes      | Yes       |

Source: `BILLING\_TIERS` constant in `supabase/functions/_shared/governance/constants.ts`. Storage limits are enforced via a PostgreSQL trigger (`enforce\_storage\_tier\_limit`). Rollover percentages are computed by `get_rollover_percentage(tier)` SQL function. XP multipliers are in `XP_MULTIPLIERS` constant in `billing-manager/index.ts`.

## 3. Tier-Dependent Infrastructure

### 3.1 Storage Limits

The SQL function ``get_storage_limit_bytes(tier)`` returns per-tier byte limits. A BEFORE INSERT/UPDATE trigger on ``user_storage`` calls ``enforce_storage_tier_limit()`` which looks up the user's tier, adds the new file size to existing usage, and raises an exception if the total exceeds the limit. This is enforced at the database level, not in application code.

|           |                          |
|-----------|--------------------------|
| Free:     | 52,428,800 B (50 MB)     |
| Builder:  | 524,288,000 B (500 MB)   |
| Operator: | 2,147,483,648 B (2 GB)   |
| Founder:  | 10,737,418,240 B (10 GB) |

### 3.2 Rollover Percentages

At each monthly billing cycle, ``process_monthly_xp_rollover()`` runs. It computes how much earned XP rolls over based on tier. Free tier users forfeit all unused earned XP. Builder retains 25%, Operator 50%, Founder 100%. The rolled-over XP moves from ``earned`` to ``rollover`` pool; the remainder is forfeited. All movements are logged in ``xp_transactions`` with balance snapshots.

### 3.3 XP Earning Multipliers

When a user completes a task or earns XP via the ``billing-manager`` 'earn' action, the base amount is multiplied by the tier multiplier before being credited to the earned pool. Free = 1x, Builder = 1.5x, Operator = 2x, Founder = 3x.

## 4. XP Economy -- Data Model

### 4.1 Table: user\_xp (one row per user)

The central XP ledger. Created automatically on signup via the `handle_new_user()` trigger.

```
user_xp (
  user_id      uuid PRIMARY KEY -> FK to users(id)
  earned       bigint NOT NULL DEFAULT 0 (acquired via tasks)
  purchased    bigint NOT NULL DEFAULT 0 (acquired via payment)
  rollover     bigint NOT NULL DEFAULT 0 (carried from previous cycle)
  reserved_xp  bigint NOT NULL DEFAULT 0 (locked for in-flight AI calls)
  created_at   timestampz
  updated_at   timestampz
  CONSTRAINT reserved_within_total:
    reserved_xp <= earned + purchased + rollover
)
```

All four balance columns have non-negative CHECK constraints. The critical invariant is:  $\text{available\_xp} = \text{earned} + \text{purchased} + \text{rollover} - \text{reserved\_xp}$ . Available XP must be  $\geq 0$  and must be  $\geq$  the amount being reserved or deducted.

### 4.2 Table: xp\_transactions (immutable audit log)

Every XP mutation is recorded as an immutable row with full balance snapshots.

```
xp_transactions (
  id              uuid PK
  user_id         uuid FK
  transaction_type: 'earn'|'purchase'|'deduct'|'reserve'|'release'|
                  'rollover_transfer'|'refund'
  amount          bigint (absolute value of operation)
  delta_rollover  bigint (change to rollover pool)
  delta_purchased bigint (change to purchased pool)
  delta_earned    bigint (change to earned pool)
  delta_reserved  bigint (change to reserved pool)
  balance_rollover bigint (post-operation snapshot)
  balance_purchased bigint
  balance_earned  bigint
  balance_reserved bigint
  reference_type  text (e.g. 'billing_cycle', 'session')
  reference_id    uuid
  idempotency_key text UNIQUE(user_id, idempotency_key)
  description     text
  metadata        jsonb
  created_at      timestampz
)
```

### 4.3 Table: xp\_reservations (in-flight locks)

Tracks active reservations with automatic TTL expiry after 15 minutes. Created during preflight, cleared during finalize or cancel. Expired reservations can be cleaned up by a background job.

```
xp_reservations (
  id          uuid PK
  user_id     uuid FK
  amount      integer
  feature     text (which feature reserved)
  status      text 'active' | 'expired' | 'released'
  expires_at  timestampz (now() + 15 min)
  created_at  timestampz
)
```

### 4.4 Table: dead\_letter\_reservations

Dead letter queue for failed cancellations requiring manual resolution. When a cancel/finalize fails (e.g., network partition with the database), the reservation details are written here so ops can reconcile.

```
dead_letter_reservations (
  id          uuid PK
```

## Opus Devia -- Tiers, XP & Billing Architecture

```
user_id    uuid FK
amount     integer
reason     text
status     text      'pending' | 'resolved'
created_at timestamptz
resolved_at timestamptz
)
```

## 5. XP Economy -- RPC Functions

All XP mutations execute as PostgreSQL RPC functions with SECURITY DEFINER. This means they run with elevated privileges regardless of the caller's role, ensuring atomicity and enforcing invariants at the database level.

### 5.1 reserve\_xp(p\_user\_id, p\_amount, ...)

Atomically locks the user\_xp row (SELECT ... FOR UPDATE), checks available XP (earned + purchased + rollover - reserved\_xp), increments reserved\_xp, and logs a 'reserve' transaction. Returns the transaction ID. Raises exception if insufficient.

```
available = rollover + purchased + earned - reserved_xp
IF available < amount -> RAISE EXCEPTION
reserved_xp = reserved_xp + amount
log_xp_transaction('reserve', amount, d__reserved=+amount)
```

### 5.2 release\_xp\_reservation(p\_user\_id, p\_amount, ...)

Reverses a reservation. Row-locks user\_xp, decrements reserved\_xp (verifying it doesn't go below 0), logs a 'release' transaction with d\_\_reserved = -amount.

### 5.3 deduct\_xp(p\_user\_id, p\_amount, p\_from\_reservation, ...)

The core spending function. Supports two modes: (1) deducting from a prior reservation (only touches reserved\_xp), or (2) direct deduction from available pools. Deduction waterfall order: rollover -> purchased -> earned. Each pool is drained up to the remaining amount before moving to the next. If any amount remains after all pools exhausted, raises exception.

```
Waterfall:
  IF rollover > 0: take = MIN(rollover, remaining), d__rollover = -take
  IF purchased > 0: take = MIN(purchased, remaining), d__purchased = -take
  IF earned > 0: take = MIN(earned, remaining), d__earned = -take
  IF remaining > 0: RAISE EXCEPTION 'pool exhaustion'
```

### 5.4 credit\_xp(p\_user\_id, p\_amount, p\_pool, ...)

Credits XP to one of three pools: 'earned', 'purchased', or 'rollover'. The pool parameter determines which column is incremented and which transaction\_type is logged ('earn', 'purchase', or 'rollover\_transfer').

### 5.5 log\_xp\_transaction(...) -- Universal Logger

Records all XP mutations with full deltas AND balance snapshots. Supports idempotency via UNIQUE(user\_id, idempotency\_key). If the same idempotency key is replayed, the function returns the existing transaction ID instead of creating a duplicate.

### 5.6 process\_monthly\_xp\_rollover() -- Cron Job

Iterates over all user\_xp rows with earned > 0. Looks up tier, computes rollover percentage. Free tier (0%): all earned XP forfeited (set to 0). Paid tiers: transfer = FLOOR(earned \* pct) moves to rollover; remainder forfeited. Logs every operation with idempotency key 'rollover-{user\_id}-{YYYY-MM}'.

## 6. billing-manager Edge Function -- XP Gateway

Location: supabase/functions/billing-manager/index.ts. This is the single HTTP entry point for all XP operations from the frontend and other edge functions. It acts as a thin coordinator that delegates heavy lifting to PostgreSQL RPCs.

### 6.1 Estimated XP Costs (ESTIMATED\_XP\_COST)

Each feature has a pre-configured estimated XP cost used during the preflight phase:

| Feature               | Est. XP | Feature                | Est. XP |
|-----------------------|---------|------------------------|---------|
| mentor_message        | 5       | assistant_message      | 2       |
| roadmap_assistant_msg | 2       | journal_assistant_msg  | 1       |
| image_upload          | 3       | roadmap_generation     | 10      |
| roadmap_recalibration | 8       | weekly_review          | 6       |
| daily_review          | 3       | monthly_breakdown      | 10      |
| voice_session_free    | 40      | voice_session_paid     | 20      |
| deep_research         | 18      | summarization          | 0       |
| memory_tagging        | 0       | journal_classification | 0       |
| intent_detection      | 0       | chat_pattern_analysis  | 0       |

### 6.2 Actions

The edge function accepts JSON with an 'action' field:

- \* preflight: Checks available XP, invokes reserve\_xp() RPC. Returns allowed=true/false.
- \* finalize: Releases reservation, deducts actual XP (ceil(tokens/2000)), invokes deduct\_xp().
- \* cancel: Releases reservation. Used when the AI call fails.
- \* earn: Looks up tier multiplier, credits earned XP pool. Used on task completion.

### 6.3 Deduction Order (deductFromPools)

The edge function also contains a JavaScript-side `deductFromPools()` that mirrors the database `deduct\_xp` RPC. This is likely a fallback/legacy path. The authoritative deduction order in both implementations is: rollover -> purchased -> earned.



## 7. XP Reservations & Dead Letter Queue

Migration: 20260606160000\_xp\_reservations\_and\_dead\_letter.sql.

### 7.1 Reservation Flow

1. preflight -> billing-manager calls reserve\_xp() RPC -> creates xp\_reservations row with 15-min TTL
2. AI call executes (may take several seconds)
  - 3a. Success -> finalize -> release\_xp\_reservation() + deduct\_xp() -> reservation cleared
  - 3b. Failure -> cancel -> release\_xp\_reservation() -> reservation cleared
  - 3c. Timeout (no response) -> reservation auto-expires at expires\_at, cleaned by cron

This pattern prevents double-spending: XP is locked (reserved) BEFORE the AI call, so two concurrent requests can't both spend the same XP.

### 7.2 Dead Letter Queue

If cancel fails (e.g., database becomes unreachable during release), the system writes to `dead\_letter\_reservations`. An ops process or manual resolution can then inspect these and reconcile. Status is 'pending' until resolved.

## 8. Billing Tier Governance (Feature Gating & Token Budgets)

Location: supabase/functions/\_shared/governance/constants.ts -- BILLING\_TIERS constant.

### 8.1 Feature Allowlist Per Tier

Each tier has an explicit `allowedFeatures` array. The function `tierAllowsFeature(tier, feature)` checks membership. This is called by the AI router (`routeAIRequest`) before any model call. If the user's tier doesn't allow the feature, a `GovernanceViolationError` is thrown.

Free tier gets only: assistant\_message, summarization, intent\_detection, journal\_classification. No mentor, no voice, no roadmap generation, no reviews. Builder adds mentor, voice, daily review, memory tagging, journal assistant, roadmap assistant. Operator and Founder add everything (image upload, roadmap gen, recalibration, all reviews, chat pattern analysis).

### 8.2 Monthly Token Budgets

Token budgets are declared per tier (50K / 150K / 400K / 1M) but enforcement is not visible in the current codebase's billing-manager. This appears to be a declared limit for future enforcement or dashboard display. The actual per-call XP cost is token-based:  $\text{actualXp} = \text{ceil}(\text{totalTokens} / 2000)$  in the finalize action.

## 9. Model Routing & Provider Architecture

Six AI providers are configured, each mapped to specific features. The function ``resolveProviderForFeature(feature)`` returns the provider, then ``getCredentialsForFeature()`` loads API keys from `Deno.env` (Supabase Edge secrets).

| Provider       | Model Backend              | Features Served                                                                    | User-Facing |
|----------------|----------------------------|------------------------------------------------------------------------------------|-------------|
| MENTOR         | DeepSeek V4 Pro            | mentor_message                                                                     | Yes         |
| ASSISTANT      | Gemini 2.5 Flash           | assistant, roadmap_asst, journal_asst, image_upload                                | Yes         |
| GEMINI         | Gemini 2.5 Flash           | roadmap_gen, recalibration, weekly/monthly/daily review                            | Yes         |
| DEEPSEEK       | DeepSeek V4 Flash          | summaries, memory_tagging, journal_classification, intent_detection, chat_patterns | No (bg)     |
| OPENAI_TTS     | OpenAI TTS (tts-1)         | voice_output                                                                       | Yes         |
| OPENAI_WHISPER | OpenAI Whisper (whisper-1) | voice_input                                                                        | Yes         |

Key architectural note: DEEPSEEK is marked ``backgroundJobsOnly: true`` and ``isUserFacing: false``. It never directly serves user requests -- only internal jobs like summarization and pattern analysis. Model secrets are loaded at runtime from `Deno.env` with support for aliases like `OPENAI_API_BASE_URL` and `OPENAI_2_API_BASE_URL`.

## 10. Frontend XP Integration

### 10.1 Profile Page (Profile.tsx)

The Profile page fetches `earned`, `purchased`, `rollover` from `user\_xp` via Supabase client. Displays a breakdown card with individual pool balances and a total. Also shows the assertiveness slider (1-5) from the `users` table, with live save on change.

### 10.2 Home Page (Home.tsx)

Fetches the full user\_xp row (earned, purchased, rollover, reserved\_xp). Computes level as  $\text{floor}(\text{earned} / 100)$  -- every 100 earned XP = 1 level. Displays `earnedXP` and `totalXP` in the LevelHero component. Also fetches streak data (current\_count, last\_activity\_date) and derives Mon-Sun weekly activity booleans for display.

### 10.3 Settings Page (Settings.tsx)

Contains an 'XP & Gamification' menu item (currently non-functional placeholder). Theme switcher with Crimson, Arctic, and Gold variants, each with custom CSS variables for glass panels, dock, accent colors, and glossy pill effects.

### 10.4 XP Estimation for Priority Tasks

The Home page estimates task XP rewards heuristically because the tasks table has no `xp\_reward` column. The heuristic: description > 200 chars = 150 XP, > 100 chars = 100 XP, else 50 XP. This is a client-side approximation; actual XP is awarded by the billing-manager 'earn' action on task completion.

## 11. Performance Metrics & Community XP Sync

### 11.1 compute\_user\_performance\_metrics(p\_user\_id)

A STABLE SQL function that computes four metrics from the user's data:

- completion\_percentage:  $\text{completed} / \text{total tasks} * 100$
- weekly\_task\_completion\_rate:  $\text{completed} / \text{total (last 7 days)} * 100$
- consistency\_rating:  $\text{current\_streak} / \text{longest\_streak} * 100$  (capped at 100)
- momentum\_score:  $(\text{completion} * 0.4) + (\text{weekly} * 0.3) + (\text{consistency} * 0.3)$

Weighted composite: 40% overall completion, 30% recency, 30% streak consistency.

### 11.2 Automatic Refresh Triggers

Three triggers auto-refresh community post metrics whenever tasks, streaks, or performance change: tasks\_refresh\_community\_metrics, user\_streaks\_refresh\_community\_metrics, user\_performance\_refresh\_community\_metrics. These cascade refresh\_user\_performance() -> refresh\_community\_post\_metrics() which updates all community\_posts rows for that user with the latest metrics.

## 12. End-to-End XP Lifecycle (Walkthrough)

Step-by-step trace of a typical user interaction:

- \* 1. User signs up -> `handle_new_user()` trigger creates users row + user\_xp row + user\_performance row
- \* 2. User completes onboarding -> `users.onboarding_complete = true`
- \* 3. User requests mentor message -> frontend calls `billing-manager?action=preflight`
- \* 4. billing-manager checks tier, computes estimate (`mentor_message = 5 XP`), calls `reserve_xp()` RPC
- \* 5. `reserve_xp()` row-locks user\_xp, checks `available >= 5`, increments `reserved_xp` by 5, logs transaction
- \* 6. billing-manager returns `{allowed: true, reservedAmount: 5}`
- \* 7. Frontend sends message to model-router edge function, which routes to MENTOR provider (DeepSeek V4 Pro)
- \* 8. AI responds with N tokens
- \* 9. Frontend calls `billing-manager?action=finalize` with `totalTokens=N` and `reservedAmount=5`
- \* 10. billing-manager computes `actualXp = ceil(N/2000)`, releases reservation, deducts actualXp
- \* 11. `deduct_xp()` waterfall: rollover -> purchased -> earned. Logs with balances.
- \* 12. User completes a task -> `billing-manager?action=earn` is called
- \* 13. earn action looks up tier multiplier, computes `xpEarned = base * multiplier`, credits earned pool
- \* 14. At month-end, `process_monthly_xp_rollover()` runs: tier-dependent rollover of earned XP

If step 8 fails (AI error): frontend calls `billing-manager?action=cancel` -> `release_xp_reservation()` -> XP returned to available pool. If cancel itself fails: reservation written to `dead_letter_reservations`.

## 13. Key Architectural Decisions & Trade-offs

- \* Dual-write reservation pattern (preflight -> finalize/cancel): Prevents double-spending in distributed AI calls. Trade-off: adds latency (extra round-trip) and requires cleanup for abandoned reservations (15-min TTL).
- \* SECURITY DEFINER RPCs: All XP mutations run with elevated DB privileges. Trade-off: bypasses RLS, so any edge function with the service\_role key can mutate XP -- trust boundary is the edge function, not the database.
- \* Immutable transaction log with balance snapshots: Every XP change records before/after balances. Enables full audit trail, reconciliation, analytics. Trade-off: more storage, more complex queries.
- \* Idempotency keys on transactions: Prevents double-charging from retries. UNIQUE constraint on (user\_id, idempotency\_key). Trade-off: callers must generate unique keys per operation.
- \* Tier enum in PostgreSQL: Tiers are database-level, not config-file. Makes them queryable in SQL functions (get\_rollover\_percentage, get\_storage\_limit\_bytes). Trade-off: adding a tier requires migration.
- \* Estimated vs actual XP: preflight uses ESTIMATED\_XP\_COST constants; finalize uses ceil(tokens/2000). Trade-off: estimates can diverge from actuals (depends on model verbosity), but simplifies preflight without knowing token count upfront.
- \* Deduction waterfall (rollover -> purchased -> earned): Rollover expires if unused, so it's consumed first. Purchased is real money, so it's consumed second. Earned (free) is consumed last.
- \* JavaScript-side deductFromPools() mirrors SQL deduct\_xp(): Duplicate implementation in billing-manager edge function and PostgreSQL RPC. Risk of divergence. The SQL RPC is authoritative (SECURITY DEFINER, row-locked), the JS version appears to be legacy.
- \* Dead letter queue: Provides safety net for failed cancellations. Currently manual resolution only -- no automated reconciliation process implemented.
- \* Monthly rollover as PostgreSQL function: Processed in-database rather than in edge function. Benefits from atomic batch operations and idempotency via composite key. Must be triggered by an external cron scheduler (pg\_cron or external).

*Report generated from kin workspace -- supabase/functions/ + supabase/migrations/ + opus-devia/src/*

*All code references are to files as they existed at generation time.*